EXPERIMENT 4 : INFORMATION RETRIEVAL USING NLTK

AIM: To design and implement an information retrieval system using NLTK, TF-IDF vectorization, and Cosine Similarity on the Amazon Fine Food Reviews dataset by preprocessing customer review text, converting text to numerical feature vectors, and retrieving the top 5 most relevant reviews for user search queries.

ALGORITHM:

1. Import required Python libraries (pandas, nltk, TfidfVectorizer from sklearn.feature_extraction.text, cosine_similarity from sklearn.metrics.pairwise) and download necessary NLTK packages (e.g., stopwords, punkt).
2. Load the Amazon Fine Food Reviews Dataset from Kaggle into a Pandas DataFrame.
3. Clean the dataset by selecting the Text column, removing null or missing values, dropping duplicate reviews, and retaining the first 10,000 reviews for faster processing.
4. Define a text preprocessing function to convert text to lowercase, strip punctuation, numbers, and special characters, tokenize text into words, remove English stopwords, and rejoin tokens into a cleaned text string.
5. Apply the preprocessing function to all customer reviews and save the processed text in a new column named Cleaned_Text.
6. Display original and cleaned versions of any five sample reviews.
7. Convert the Cleaned_Text column into TF-IDF feature vectors using TfidfVectorizer and display the resulting matrix shape and total vocabulary size.
8. Define an Information Retrieval function that:
    - Accepts an input search query.
    - Preprocesses the query using the text preprocessing pipeline.
    - Transforms the query into a TF-IDF vector.
    - Computes Cosine Similarity between the query vector and all document vectors.
    - Ranks and extracts the Top 5 most similar reviews displaying similarity scores, original review text, and cleaned review text.

9. Evaluate the retrieval system using sample test queries: "great product with fast shipping", "disappointed", "excellent taste", and "poor packaging".
10. Compare the retrieved results across queries to verify relevance.

Information Retrieval (IR) using NLTK

Step 1: Import Libraries and Download NLTK Resources

```
#Information_Retrival _2
#Sanjay S(241801246)
import nltk
import pandas as pd
import string
import re
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

Step 2: Load the 20 Newsgroups Dataset

```
#Information_Retrival _2
#SANJAY_S_241801246
newsgroups_data = fetch_20newsgroups(
    subset='train',
    remove=('headers', 'footers', 'quotes')
)
documents = newsgroups_data.data[:1000]
df = pd.DataFrame({'text': documents})
print(df.head(2).to_markdown())
```

```
|    | text                                                
```

Step 3: Load and Extract Stop Words

```
#Information_Retrival _2
#SANJAY_S_241801246
stop_words = set(stopwords.words('english'))
print("Sample Stop Words:")
print(list(stop_words)[:5])
```

```
Sample Stop Words:
['them', "they'll", 'doesn', 'so', 'whom']
```

Step 4: Extract Stop Words from Text

```
#Information_Retrival _2
#SANJAY_S_241801246
def extract_stop_words(text):
    text = text.lower()
    tokens = word_tokenize(text)
    return [word for word in tokens if word in stop_words and word.isalpha()]
stop_words_list = df['text'].apply(extract_stop_words)
stop_words_list
```

	text
0	[i, was, if, out, there, me, on, this, i, the,...
1	[a, of, who, their, have, their, for, this, a,...
2	[my, up, the, this, after, as, a, in, i, in, t...
3	[do, you, have, i, to, some, about, this]
4	[from, by, a, my, is, that, the, are, in, the,...
...	...
995	[i, have, a, and, with, an]
996	[you, do, they, have, out, y, of, to, the, me,...

Step 5: Preprocess the Text

```
#Information_Retrival _2
#SANJAY_S_241801246
def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    tokens = word_tokenize(text)
    tokens = [w for w in tokens if w not in stop_words]
    return ' '.join(tokens)
df['clean_text'] = df['text'].apply(preprocess)
df['clean_text']
```

...

	clean_text
0	wondering anyone could enlighten car saw day d...
1	fair number brave souls upgraded si clock osci...
2	well folks mac plus finally gave ghost weekend...
3	weiteks addressphone number id like get inform...
4	article cowcbnpworldstdcom tombakerworldstdcom...
...	...
995	vt vt compatible terminal external hyess modem...
996	obviously really dont know spoken cf secy stat...
997	noticed exact phenomenon occurs lciii perhaps ...
998	essential tremor progressive hereditary tremor...
999	sorry posting email keeps bouncing maybe help ...

Step 6: TF-IDF Vectorization

```
#Information_Retrival _2
#SANJAY_S_241801246
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(df['clean_text'])
```

Step 7: Define the Search Query

```
#Information_Retrival _2
#SANJAY_S_241801246
query = "space shuttle launch"
query_clean = preprocess(query)
query_vec = vectorizer.transform([query_clean])
```

Step 8: Compute Cosine Similarity

```
#Information_Retrival _2
#SANJAY_S_241801246
similarities = cosine_similarity(
    query_vec,
    tfidf_matrix
).flatten()
print(similarities[:5])
```

```
[0.         0.         0.         0.         0.10156444]
```

Step 9: Rank the Top 5 Documents

Step 9: Rank the Top 5 Documents

```
#Information_Retrival _2
#SANJAY_S_241801246
top_indices = similarities.argsort()[-5:][::-1]
print("\nTop 5 Relevant Documents for Query:", query)
for i in top_indices:
    print(
        f"\nDoc {i+1} | Score: {similarities[i]:.4f}\n"
        f"{df.loc[i, 'text'][:400]}...\n"
    )
```

```
Doc 154 | Score: 0.3939
Archive-name: space/schedule
Last-modified: $Date: 93/04/01 14:39:23 $

SPACE SHUTTLE ANSWERS, LAUNCH SCHEDULES, TV COVERAGE

    SHUTTLE LAUNCHINGS AND LANDINGS; SCHEDULES AND HOW TO SEE THEM

    Shuttle operations are discussed in the Usenet group sci.space.shuttle,
    and Ken Hollis (gandalf@pro-electric.cts.com) posts a compressed version
    of the shuttle manifest (launch dates and other i...


Doc 344 | Score: 0.2446

In fact, you probably want to avoid US Government anything for such a
project.  The pricetag is invariably too high, either in money or in
hassles.

The important thing to realize here is that the big cost of getting to
the Moon is getting into low Earth orbit.  Everything else is practically
down in the noise.  The only part of getting to the Moon that poses any
new problems, beyond what you fac...


Doc 60 | Score: 0.2308
Archive-name: space/new_probes
Last-modified: $Date: 93/04/01 14:39:17 $

UPCOMING PLANETARY PROBES - MISSIONS AND SCHEDULES

    Information on upcoming or currently active missions not mentioned below
    would be welcome. Sources: NASA fact sheets, Cassini Mission Design
    team, ISAS/NASDA launch schedules, press kits.


    ASUKA (ASTRO-D) - ISAS (Japan) X-ray astronomy satellite, launched i...


Doc 357 | Score: 0.2017
According to a Software engineering professor here, what was actually rated
level five was an ibm unit which produced part of the software for the shuttle,
by not means all of it.

Interesting note: 90% of the software development groups surveyed were at
level 1. The ibm shuttle groups was the *only* one at level 5!
...


Doc 233 | Score: 0.1982


SPECIFIC:
```

Step 10: POS Tagging

```
#Information_Retrival _2
#SANJAY_S_241801246
for i in range(3):
    tokens = word_tokenize(df.loc[i, 'clean_text'])
    tagged = nltk.pos_tag(tokens)
    print(f"\nDocument {i+1} POS Tags:")
    print(tagged[:10])
```

```
Document 1 POS Tags:
[('wondering', 'VBG'), ('anyone', 'NN'), ('could', 'MD'), ('enlighten', 'VB'), ('car', 'NN'), ('saw', 'JJ'), ('day', 'NN'), ('door', 'VB'), ('sports', 'NNS'), ('car', 'NN')]

Document 2 POS Tags:
[('fair', 'JJ'), ('number', 'NN'), ('brave', 'VBP'), ('souls', 'NNS'), ('upgraded', 'VBD'), ('si', 'JJ'), ('clock', 'NN'), ('oscillator', 'NN'), ('shared', 'VBD'), ('experiences', 'NNS')]

Document 3 POS Tags:
[('well', 'RB'), ('folks', 'NNS'), ('mac', 'VBP'), ('plus', 'CC'), ('finally', 'RB'), ('gave', 'VBD'), ('ghost', 'VBN'), ('weekend', 'NN'), ('starting', 'VBG'), ('life', 'NN')]
```

RESULT: An Information Retrieval system using NLTK, TF-IDF, and Cosine Similarity was successfully built and evaluated. Customer reviews were cleaned, preprocessed, vectorized, and the system effectively retrieved and ranked the top 5 relevant reviews with similarity scores for various search queries.